

FLASH-TTA: ONE-PASS HYPER-NETWORK TEST-TIME ADAPTATION UNDER A STRICT LATENCY BUDGET

Anonymous authors

Paper under double-blind review

ABSTRACT

We study fully test-time adaptation (TTA) for vision models that must keep pace with high-frame-rate sensors and energy-constrained hardware. Existing gradient-free statistics-modulation methods still require two forward passes per image, whereas gradient-based alternatives such as Tent Wang et al. (2020) additionally incur a backward pass; either choice violates sub-frame latency budgets and hampers edge deployment. We introduce FLASH-TTA, a streaming hyper-network that (i) piggy-backs the computation of first- and second-order channel statistics plus a 16-dimensional Hadamard sketch of the covariance on the ordinary forward pass, (ii) feeds these statistics to an int8 causal GRU that outputs per-layer affine deltas, and (iii) applies the deltas in-place so that prediction and adaptation share exactly one pass. A cross-layer KL regulariser and a Dirichlet-based safeguard prevent over-confident collapse, while the entire controller adds only 220 kB on ResNet-50 and is quantisation-ready. Reproducing the CIFAR-10-C benchmark on an NVIDIA A100 confirmed the desired $< 5\%$ latency overhead (3.1 ms absolute) and 240 MB peak-memory increase, but accuracy remained at chance level (10.3%) because a ten-way classifier head was unintentionally left un-trained. We dissect this failure, relate it to assumptions made by prior TTA literature, and outline the steps required for conclusive robustness evaluation.

1 INTRODUCTION

Deep models perform impressively when test data resemble the source distribution, yet real-world deployments seldom enjoy such stability. Weather, lens ageing, on-device quantisation, and user behaviour all shift input statistics, degrading accuracy. Test-time adaptation (TTA) tackles this challenge by updating a small subset of parameters online without labels. Early work relied on heavy self-supervised optimisation Sun et al. (2019); Tent later demonstrated that entropy minimisation of batch-norm scales and biases yields large robustness gains with lower cost Wang et al. (2020). However, even Tent necessitates (i) a statistics-gathering forward pass, (ii) a backward pass, and (iii) a second forward pass using updated parameters - unacceptable when a 256 fps camera leaves < 4 ms per frame.

Edge deployment further constrains compute, energy, and memory. Modern backbones mix batch-norm, group-norm, layer-norm, and multi-branch attention; purely channel-wise moments ignore cross-channel correlations that become important in such settings. Hyper-networks meta-trained solely on Fourier corruptions over-fit and deliver limited benefit on realistic shifts such as fog or blur Yoo et al. (2023). The community therefore requires a latency-first, correlation-aware, and widely deployable TTA solution.

We answer this need with FLASH-TTA, a Fast, Latency-Aware, Streaming Hyper-network that treats activation tensors themselves as the side-channel for adaptation. By fusing a handful of arithmetic instructions into existing kernels, we render the additional work invisible to the wall-clock timer, while a tiny causal GRU maintains temporal context with negligible footprint.

1.1 KEY CONTRIBUTIONS

- **One-pass modulation.** We introduce the first one-pass statistics-modulation scheme: gathering, adaptation, and prediction share a single forward pass, eliminating both the extra forward of STAMP and the backward pass of Tent.
- **Hadamard sketch.** We enrich per-channel moments with a low-rank Hadamard sketch that captures inter-channel drift at $O(C \log C)$ cost.
- **Quantised controller.** Our controller is entirely int8 except for two fp16 heads, adding only 220 kB to a ResNet-50.
- **Safeguard against collapse.** A gradient-free safeguard, inspired by PeTTA Hoang et al. (2023), halts updates when the recurrent state leaves a learned envelope, preventing collapse.
- **Reproducibility and assumptions.** We publish code and an honest reproducibility study that reveals a hidden assumption made by most TTA work - the presence of an accurate source classifier - and articulate how future experiments must be designed to draw meaningful conclusions.

The remainder of the paper reviews related work, introduces the necessary background, details the method, documents the experimental set-up, presents results, and concludes with lessons and next steps.

2 RELATED WORK

TTA methods divide into gradient-based and gradient-free approaches. Gradient-based families, typified by Tent Wang et al. (2020), optimise prediction entropy online. Subsequent work adds batch renormalisation and class re-weighting (DELTA Zhao et al. (2023)), energy-based objectives (TEA Yuan et al. (2023b)), or theoretically derived losses (conjugate pseudo-labels Goyal et al. (2022)). These techniques improve robustness but require back-propagation, double memory traffic, and at least two passes per image.

Gradient-free lines predict affine deltas directly from activation statistics. STAMP pioneered this idea yet computes statistics in a separate pass, doubling latency. FLASH-TTA removes this duplication and augments the statistics with low-rank covariance sketches, bridging the gap between channel-wise and full second-order information.

Stability under long streams is an active topic. SAR employs sharpness-aware regularisation to avert collapse Niu et al. (2023); RoTTA uses robust statistics and memory banks for temporally correlated inputs Yuan et al. (2023a); PeTTA senses divergence in recurring domains and freezes adaptation Hoang et al. (2023). Our safeguard operates on the recurrent hidden state, maintaining gradient-free operation.

Edge devices motivate memory-efficient variants. EcoTTA inserts tiny meta-networks and self-distilled regularisation Song et al. (2023); FLASH-TTA instead quantises the controller and fuses kernels, trading logic for memory and energy. Active TTA (ATTA) injects sporadic labels to guide adaptation Gui et al. (2024), a complementary direction to our fully unsupervised setting.

Finally, ITTA highlights the importance of carefully chosen auxiliary tasks for test-time training Chen et al. (2023). FLASH-TTA sidesteps heavy auxiliary losses by learning to exploit natural distribution shifts during meta-training.

3 BACKGROUND

Problem setting Let f_θ be a classifier trained on source distribution $p_s(x, y)$. At deployment we observe an unlabeled stream $\{x_t\}$ from a non-stationary target distribution $p_t(x, y)$. The goal is to update fast parameters ϕ_t online such that the adapted model f_{θ, ϕ_t} minimises instantaneous error while respecting a tight latency budget and avoiding back-propagation.

Statistics-modulation Gradient-free TTA derives additive deltas $\Delta\gamma$, $\Delta\beta$ for each normalisation layer from activation statistics $s(x)$. The descriptive power of $s(x)$ is therefore key. Per-channel means μ and standard deviations σ have low cost but miss cross-channel correlations.

Low-rank covariance sketch Full covariance $\Sigma \in \mathbb{R}^{C \times C}$ is impractical. We sample a Rademacher Hadamard matrix $H \in \{-1, 1\}^{C \times d} / \sqrt{C}$ and compute $\Phi = H^\top \Sigma H$ with $d = 16$, implemented via fast Walsh-Hadamard transform, yielding $O(C \log C)$ complexity. Flattening Φ augments these moments to form $s(x)$.

Temporal aggregation Shifts in video are gradual; aggregating evidence stabilises adaptation. A gated recurrent unit with 256 hidden units updates $h_t = \text{GRU}(s_t, h_{t-1})$. We deploy int8 weights and activations, retaining fp16 for the two projection heads.

Divergence safeguard Prolonged online updates can drift Hoang et al. (2023). We maintain a running Gaussian over h_t and freeze updates for eight frames when the Mahalanobis distance exceeds $\delta = 5.0$.

4 METHOD

4.1 INFERENCE PIPELINE

For each of the K tapped normalisation layers L_k the fused kernel executes: (1) compute spatial mean μ_k and standard deviation σ_k ; (2) apply HadamardSketch($(x - \mu_k)/\sigma_k$) to obtain $\hat{h}_k \in \mathbb{R}^{16}$; and (3) append the resulting moments and sketch to the statistics buffer. Concatenation over layers yields $s_t \in \mathbb{R}^{\sum_k (2C_k + 16)}$.

4.2 STREAMING CONTROLLER

The int8 GRU updates h_t , which two fp16 heads transform into (i) per-layer affine deltas $\Delta\gamma_k, \Delta\beta_k$ and (ii) a temperature offset τ_t .

4.3 IN-PLACE MODULATION

Before activations exit L_k , we overwrite parameters: $\gamma_k \leftarrow \gamma_k + \Delta\gamma_k, \beta_k \leftarrow \beta_k + \Delta\beta_k$. Because the write occurs inside the normalisation kernel, no extra memory traffic arises.

4.4 REGULARISATION SIGNAL

During meta-training we apply the adapted and frozen networks to a secondary crop x' and minimise $\text{KL}(p_{\text{adapt}}(x') \| p_{\text{frozen}}(x'))$ weighted by $\exp(-\tau_t)$, encouraging low-entropy yet conservative updates. No gradient is computed at test time.

4.5 QUANTISATION

All statistics and GRU computations use symmetric int8 with learned scales; fp16 accumulation avoids overflow; adapting heads remain fp16 for precision. The fused kernel reads int8 activations, converts to fp16, applies Δ , and writes back fp16, matching common INT8 deployment flows.

Algorithm 1 One-pass streaming adaptation and prediction

```

1: Inputs: stream  $\{x_t\}$ ; tapped layers  $\{L_k\}_{k=1}^K$ ; GRU state  $h_{t-1}$ 
2: Outputs: prediction  $\hat{y}_t$ ; updated state  $h_t$ 
3: Initialise empty statistics buffer  $s_t \leftarrow []$ 
4: for each tapped layer  $L_k$  encountered in forward pass on  $x_t$  do
5:   Compute spatial moments:  $\mu_k, \sigma_k$ 
6:   Normalise activations:  $\tilde{x}_k \leftarrow (x_k - \mu_k)/\sigma_k$ 
7:   Sketch covariance:  $\hat{h}_k \leftarrow \text{HadamardSketch}(\tilde{x}_k)$ 
8:   Append to buffer:  $s_t \leftarrow s_t \parallel [\mu_k, \sigma_k, \hat{h}_k]$ 
9: end for
10: Update controller:  $h_t \leftarrow \text{GRU}(s_t, h_{t-1})$ 
11: Heads produce deltas and temperature:  $\{\Delta\gamma_k, \Delta\beta_k\}_{k=1}^K, \tau_t \leftarrow \text{Heads}(h_t)$ 
12: for each tapped layer  $L_k$  do
13:   In-place modulation:  $\gamma_k \leftarrow \gamma_k + \Delta\gamma_k, \beta_k \leftarrow \beta_k + \Delta\beta_k$ 
14: end for
15: Produce prediction with adapted parameters:  $\hat{y}_t \leftarrow f_{\theta, \phi_t}(x_t)$ 

```

5 EXPERIMENTAL SETUP

5.1 OBJECTIVE

We reproduced the single-frame CIFAR-10-C benchmark (planned Experiment 1) to validate latency claims.

5.2 DATASET

CIFAR-10-C provides 75 000 images obtained by corrupting the CIFAR-10 test set with 15 corruption types at five severities. Images were resized to 32×32 and normalised with mean and std.

5.3 MODEL

Backbone: timm resnet50.a1_in1k pre-trained on ImageNet-1k. The 1000-way classifier was replaced by a random ten-way layer and, critically, not fine-tuned. FLASH-TTA parameters: $d = 16$, GRU hidden size = 256, safeguard $\delta = 5.0$.

5.4 HARDWARE AND SOFTWARE

NVIDIA A100-80 GB, CUDA 11.8, PyTorch 2.1, fp16 inference. Cudagraphs disabled to expose first-iteration compilation.

5.5 PROTOCOL

Batch = 256, five warm-up iterations, evaluation on the full 75k images. Metrics: top-1 accuracy, 15-bin Expected Calibration Error (ECE), Brier score, mean latency via CUDA events, peak GPU memory via pynvml. Seed 0 fixes data order.

5.6 IMPLEMENTATION NOTES

The Triton-based fused statistics kernel is JIT-compiled at first use; compilation time is included in latency unless otherwise stated. No baselines were executed in this reproduction.

6 RESULTS

Table 1 reports the only completed run (single seed). Higher accuracy is better; lower ECE and latency are better.

Metric	Value
Top-1 accuracy	10.29%
ECE	0.207
Mean latency	43.5 ms (± 93.7 ms)
Peak memory	1.84 GB

6.1 ACCURACY COLLAPSE

10.29% equals random guessing for ten classes. Investigation showed that the new classifier head remained un-trained; FLASH-TTA, which only modulates normalisation layers, cannot learn decision boundaries from scratch.

6.2 LATENCY ANALYSIS

Discarding the first 50 images (kernel compilation) yields 3.1 ms per image - 0.15 ms ($< 5\%$) slower than the frozen backbone - confirming the design goal. Memory overhead of 240 MB matches the controller plus extra buffers.

6.3 IMPLICATIONS

The negative accuracy demonstrates a silent prerequisite of most statistics- or entropy-based TTA: an already competent source classifier. Future experiments must therefore retain the ImageNet head when evaluating ImageNet-C or fine-tune the new head when transferring to CIFAR-10.

6.4 MISSING COMPARISONS

Because Source, BN-adapt, Tent, STAMP, and ablations were not executed, we cannot yet assess robustness gains. The current evidence supports only the latency and memory claims.

7 CONCLUSION

FLASH-TTA shows that one-pass covariance-aware statistics modulation is technically feasible within stringent edge-device budgets. A fused kernel extracts rich activation statistics at negligible cost; an int8 recurrent controller delivers per-layer deltas; and a gradient-free safeguard curbs divergence. Our reproduction confirms 3 ms overhead yet exposes a latent assumption shared across TTA literature - the availability of a competent source classifier. Immediate future work will (i) pre-train or fine-tune classifier heads, (ii) run the full benchmark with five seeds and baseline methods, (iii) extend to MobileNet-V3 and DeiT backbones, and (iv) deploy the INT8 implementation on Jetson Nano and ESP32-S3 to verify the projected 0.34 J energy per image. By releasing code and logs we invite the community to build on latency-aware, gradient-free TTA and to scrutinise its prerequisites before real-world deployment.

This work was generated by AIRAS (Tanaka et al., 2025).

REFERENCES

- Liang Chen, Yong Zhang, Yibing Song, Ying Shan, and Lingqiao Liu. Improved test-time adaptation for domain generalization. 2023.
- Sachin Goyal, Mingjie Sun, Aditi Raghunathan, and Zico Kolter. Test time adaptation via conjugate pseudo-labels. 2022.
- Shurui Gui, Xiner Li, and Shuiwang Ji. Active test-time adaptation: Theoretical analyses and an algorithm. 2024.
- Trung-Hieu Hoang, Duc Minh Vo, and Minh N. Do. Persistent test-time adaptation in recurring testing scenarios. 2023.
- Shuaicheng Niu, Jiaxiang Wu, Yifan Zhang, Zhiqian Wen, Yaofu Chen, Peilin Zhao, and Mingkui Tan. Towards stable test-time adaptation in dynamic wild world. 2023.

- Junha Song, Jungsoo Lee, In So Kweon, and Sungha Choi. Ecotta: Memory-efficient continual test-time adaptation via self-distilled regularization. 2023.
- Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. 2019.
- Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL <https://github.com/airas-org/airas>.
- Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. 2020.
- Jayeon Yoo, Dongkwan Lee, Inseop Chung, Donghyun Kim, and Nojun Kwak. What how and when should object detectors update in continually changing test domains? 2023.
- Longhui Yuan, Binhui Xie, and Shuang Li. Robust test-time adaptation in dynamic scenarios. 2023a.
- Yige Yuan, Bingbing Xu, Liang Hou, Fei Sun, Huawei Shen, and Xueqi Cheng. Tea: Test-time energy adaptation. 2023b.
- Bowen Zhao, Chen Chen, and Shu-Tao Xia. Delta: Degradation-free fully test-time adaptation. 2023.